

Группа	Р3219	К работе допущен	
Студент	Ануфриев А.С.	Работа выполнена	
Преподаватель	Зинченко А.А.	Отчет принят	

Вычислительная математика

Отчёт по лабораторной работе № 4

**Аппроксимация функции методом наименьших
квадратов**

Вариант 2

2026 год
Санкт-Петербург

Содержание

1	Цель работы	3
2	Рабочие формулы метода	3
2.1	Постановка задачи аппроксимации	3
2.2	Полиномиальная аппроксимация	3
2.2.1	Линейная аппроксимация ($m = 1$): $\varphi(x) = ax + b$	3
2.2.2	Квадратичная аппроксимация ($m = 2$): $\varphi(x) = a_0 + a_1x + a_2x^2$	3
2.3	Нелинейные функции (линеаризация)	4
2.4	Метрики качества	4
3	Вычислительная часть	4
3.1	Таблица значений функции	4
3.2	Линейная аппроксимация	5
3.3	Квадратичная аппроксимация	5
3.4	Сравнение линейной и квадратичной моделей	6
3.5	Графики	7
4	Программная реализация	7
4.1	Описание программы	7
4.2	Листинг программы	7
4.3	Результаты выполнения программы	9
4.3.1	Тест 1: Данные варианта 2	9
4.3.2	Тест 2: Линейные данные	10
4.3.3	Тест 3: Некорректные данные	10
5	Выводы	11

1 Цель работы

Найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов.

Вариант 2. Функция:

$$y = \frac{15x}{x^4 + 2}, \quad x \in [0, 4], \quad h = 0,4.$$

2 Рабочие формулы метода

2.1 Постановка задачи аппроксимации

Пусть дана таблица значений (x_i, y_i) , $i = 1, \dots, n$. Требуется найти функцию $\varphi(x)$, которая наилучшим образом приближает исходные данные.

В методе наименьших квадратов (МНК) ищется функция, минимизирующая сумму квадратов отклонений:

$$S = \sum_{i=1}^n \varepsilon_i^2 = \sum_{i=1}^n [\varphi(x_i) - y_i]^2 \longrightarrow \min.$$

2.2 Полиномиальная аппроксимация

Аппроксимирующая функция — многочлен степени m :

$$\varphi(x) = a_0 + a_1x + a_2x^2 + \dots + a_mx^m.$$

Коэффициенты $a_0, \dots, a_m$ находятся из системы нормальных уравнений:

$$\begin{pmatrix} n & \sum x_i & \sum x_i^2 & \dots & \sum x_i^m \\ \sum x_i & \sum x_i^2 & \sum x_i^3 & \dots & \sum x_i^{m+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum x_i^m & \sum x_i^{m+1} & \sum x_i^{m+2} & \dots & \sum x_i^{2m} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \\ \vdots \\ \sum x_i^m y_i \end{pmatrix}.$$

2.2.1 Линейная аппроксимация ($m = 1$): $\varphi(x) = ax + b$

Система двух уравнений:

$$\begin{cases} aS_{XX} + bS_X = S_{XY}, \\ aS_X + bn = S_Y, \end{cases}$$

где $S_X = \sum x_i$, $S_{XX} = \sum x_i^2$, $S_Y = \sum y_i$, $S_{XY} = \sum x_i y_i$.

По правилу Крамера:

$$\Delta = S_{XX} \cdot n - S_X^2, \quad a = \frac{S_{XY} \cdot n - S_X \cdot S_Y}{\Delta}, \quad b = \frac{S_{XX} \cdot S_Y - S_X \cdot S_{XY}}{\Delta}.$$

2.2.2 Квадратичная аппроксимация ($m = 2$): $\varphi(x) = a_0 + a_1x + a_2x^2$

Система трёх уравнений:

$$\begin{cases} a_0n + a_1 \sum x_i + a_2 \sum x_i^2 = \sum y_i, \\ a_0 \sum x_i + a_1 \sum x_i^2 + a_2 \sum x_i^3 = \sum x_i y_i, \\ a_0 \sum x_i^2 + a_1 \sum x_i^3 + a_2 \sum x_i^4 = \sum x_i^2 y_i. \end{cases}$$

2.3 Нелинейные функции (линеаризация)

- **Экспоненциальная:** $\varphi(x) = ae^{bx}$. Линеаризация: $\ln \varphi = \ln a + bx$, далее линейный МНК по $(x, \ln y)$. Возврат: $a = e^A$, $b = B$.
- **Степенная:** $\varphi(x) = ax^b$. Линеаризация: $\ln \varphi = \ln a + b \ln x$, МНК по $(\ln x, \ln y)$. Возврат: $a = e^A$, $b = B$.
- **Логарифмическая:** $\varphi(x) = a \ln x + b$. Линеаризация: $X = \ln x$, МНК по (X, y) .

2.4 Метрики качества

Среднеквадратическое отклонение:

$$\delta = \sqrt{\frac{\sum_{i=1}^n (\varphi(x_i) - y_i)^2}{n}}.$$

Наилучшая аппроксимирующая функция — та, для которой δ минимально.

Коэффициент детерминации:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \varphi(x_i))^2}{\sum_{i=1}^n (y_i - \bar{\varphi})^2}, \quad \bar{\varphi} = \frac{1}{n} \sum_{i=1}^n \varphi(x_i).$$

Чем ближе R^2 к 1, тем лучше аппроксимация.

Коэффициент корреляции Пирсона (для линейной зависимости):

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \cdot \sum_{i=1}^n (y_i - \bar{y})^2}}, \quad -1 \leq r \leq 1.$$

3 Вычислительная часть

3.1 Таблица значений функции

Функция $y = \frac{15x}{x^4 + 2}$ на $[0, 4]$ с шагом $h = 0,4$ (11 точек):

i	x_i	y_i
0	0.0000	0.000000
1	0.4000	2.962085
2	0.8000	4.980080
3	1.2000	4.418696
4	1.6000	2.805836
5	2.0000	1.666667
6	2.4000	1.023379
7	2.8000	0.661776
8	3.2000	0.449196
9	3.6000	0.317719
10	4.0000	0.232558

Таблица 1: Таблица значений $y = 15x/(x^4 + 2)$

Функция имеет максимум вблизи $x \approx 0,8$ и монотонно убывает до нуля.

3.2 Линейная аппроксимация

Ищем $\varphi(x) = ax + b$. Вычислим необходимые суммы по $n = 11$ точкам:

$$\begin{aligned} S_X &= \sum_{i=0}^{10} x_i = 22,0000, & S_{XX} &= \sum_{i=0}^{10} x_i^2 = 61,6000, \\ S_Y &= \sum_{i=0}^{10} y_i = 19,5180, & S_{XY} &= \sum_{i=0}^{10} x_i y_i = 26,1145. \end{aligned}$$

Вычислим определители:

$$\Delta = S_{XX} \cdot n - S_X^2 = 61,6 \cdot 11 - 22^2 = 677,6 - 484 = 193,6,$$

$$\Delta_1 = S_{XY} \cdot n - S_X \cdot S_Y = 26,1145 \cdot 11 - 22 \cdot 19,5180 = 287,260 - 429,396 = -142,136,$$

$$\Delta_2 = S_{XX} \cdot S_Y - S_X \cdot S_{XY} = 61,6 \cdot 19,5180 - 22 \cdot 26,1145 = 1202,309 - 574,519 = 627,790.$$

$$a = \frac{\Delta_1}{\Delta} = \frac{-142,136}{193,6} \approx -0,7342, \quad b = \frac{\Delta_2}{\Delta} = \frac{627,790}{193,6} \approx 3,2427.$$

$$\boxed{\varphi_1(x) = -0,7342x + 3,2427}$$

Коэффициент корреляции Пирсона: $r = -0,5535$ — связь умеренная (отрицательная), что говорит о том, что линейная модель плохо описывает данную функцию (у неё максимум в начале интервала).

Таблица отклонений для линейной модели:

i	x_i	y_i	$\varphi_1(x_i)$	ε_i
0	0.0000	0.000000	3.242709	3.242709
1	0.4000	2.962085	2.949040	-0.013045
2	0.8000	4.980080	2.655371	-2.324709
3	1.2000	4.418696	2.361701	-2.056995
4	1.6000	2.805836	2.068032	-0.737804
5	2.0000	1.666667	1.774363	0.107696
6	2.4000	1.023379	1.480694	0.457315
7	2.8000	0.661776	1.187024	0.525248
8	3.2000	0.449196	0.893355	0.444159
9	3.6000	0.317719	0.599686	0.281967
10	4.0000	0.232558	0.306016	0.073458

Таблица 2: Линейная аппроксимация

$$S_{\text{лин}} = \sum \varepsilon_i^2 = 21,474, \quad \delta_{\text{лин}} = \sqrt{S/n} = \sqrt{21,474/11} \approx 1,397.$$

$$R_{\text{лин}}^2 = 0,306 \Rightarrow \text{точность аппроксимации недостаточна.}$$

3.3 Квадратичная аппроксимация

Ищем $\varphi(x) = a_0 + a_1x + a_2x^2$. Дополнительные суммы:

$$\sum x_i^3 = 193,600, \quad \sum x_i^4 = 648,525, \quad \sum x_i^2 y_i = 47,395.$$

Система:

$$\begin{cases} 11 a_0 + 22 a_1 + 61,6 a_2 = 19,518, \\ 22 a_0 + 61,6 a_1 + 193,6 a_2 = 26,115, \\ 61,6 a_0 + 193,6 a_1 + 648,525 a_2 = 47,395. \end{cases}$$

Решение системы (методом Гаусса):

$$a_0 = 2,1260, \quad a_1 = 1,1270, \quad a_2 = -0,4653.$$

$$\varphi_2(x) = 2,1260 + 1,1270 x - 0,4653 x^2$$

Таблица отклонений для квадратичной модели:

i	x_i	y_i	$\varphi_2(x_i)$	ε_i
0	0.0000	0.000000	2.126022	2.126022
1	0.4000	2.962085	2.502365	-0.459720
2	0.8000	4.980080	2.729817	-2.250263
3	1.2000	4.418696	2.808376	-1.610320
4	1.6000	2.805836	2.738044	-0.067792
5	2.0000	1.666667	2.518821	0.852154
6	2.4000	1.023379	2.150706	1.127327
7	2.8000	0.661776	1.633699	0.971923
8	3.2000	0.449196	0.967801	0.518605
9	3.6000	0.317719	0.153011	-0.164708
10	4.0000	0.232558	-0.810671	-1.043229

Таблица 3: Квадратичная аппроксимация

$$S_{\text{кв}} = 16,719, \quad \delta_{\text{кв}} = \sqrt{16,719/11} \approx 1,233.$$

$$R_{\text{кв}}^2 = 0,460 \quad \Rightarrow \quad \text{точность аппроксимации недостаточна.}$$

3.4 Сравнение линейной и квадратичной моделей

Модель	S	δ	R^2
Линейная	21.474	1.397	0.306
Квадратичная	16.719	1.233	0.460

Таблица 4: Сравнение линейной и квадратичной аппроксимаций

Квадратичная модель лучше (δ меньше), однако обе модели дают недостаточную точность, поскольку исходная функция имеет выраженный «горб» в начале интервала, что не может быть хорошо аппроксимировано прямой линией или параболой с положительной ветвью.

3.5 Графики

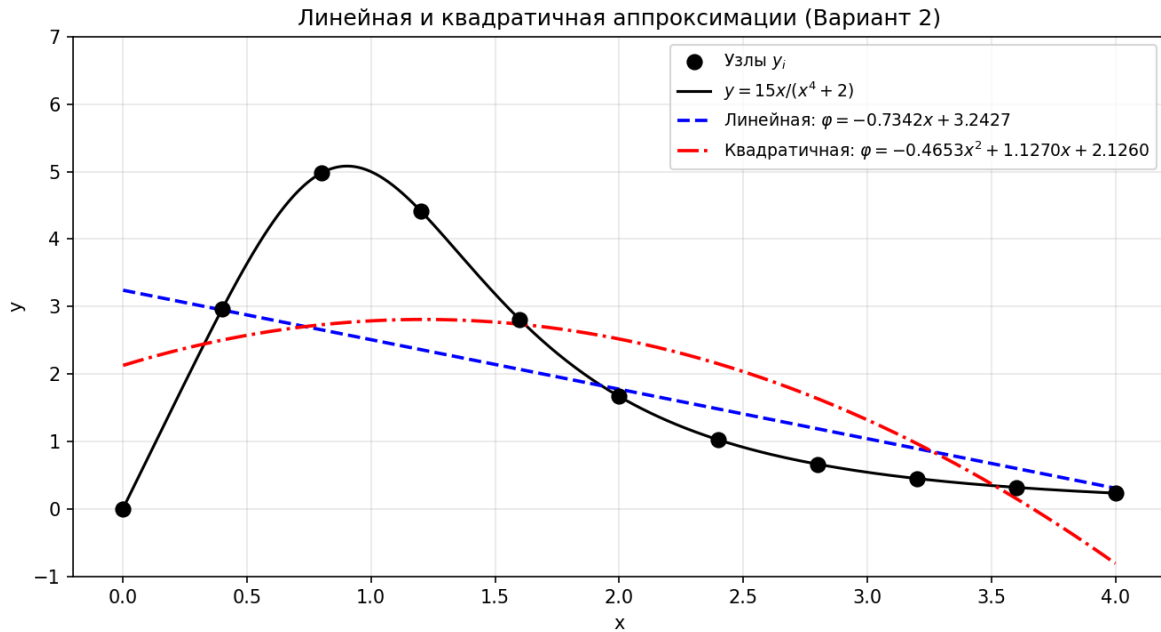


Рис. 1: Линейная и квадратичная аппроксимации

4 Программная реализация

4.1 Описание программы

Программа реализована на языке Python в виде консольного приложения с интерактивным меню. Пользователь может выбрать тип аппроксимации, загрузить данные (вариант 2 по умолчанию, ручной ввод или файл), получить коэффициенты, метрики и графики.

Реализованные методы аппроксимации:

- Линейная: $\varphi(x) = ax + b$
- Квадратичная: $\varphi(x) = a_0 + a_1x + a_2x^2$
- Кубическая: $\varphi(x) = a_0 + a_1x + a_2x^2 + a_3x^3$
- Экспоненциальная: $\varphi(x) = ae^{bx}$
- Логарифмическая: $\varphi(x) = a \ln x + b$
- Степенная: $\varphi(x) = ax^b$

Для решения систем нормальных уравнений реализован метод Гаусса с выбором ведущего элемента. Все аппроксимирующие функции написаны самостоятельно (без `numpy.polyfit` и аналогов).

4.2 Листинг программы

Листинг 1: Решение СЛАУ методом Гаусса

```
1 def gauss_solve(A, b):
2     n = len(b)
3     M = [[A[i][j] for j in range(n)] + [b[i]] for i in range(n)]
4     for col in range(n):
```

```

5     max_row = max(range(col, n), key=lambda r: abs(M[r][col]))
6     M[col], M[max_row] = M[max_row], M[col]
7     if abs(M[col][col]) < 1e-15:
8         raise ValueError("Singular matrix")
9     for row in range(col + 1, n):
10        factor = M[row][col] / M[col][col]
11        for j in range(col, n + 1):
12            M[row][j] -= factor * M[col][j]
13    x = [0.0] * n
14    for i in range(n - 1, -1, -1):
15        x[i] = M[i][n]
16        for j in range(i + 1, n):
17            x[i] -= M[i][j] * x[j]
18        x[i] /= M[i][i]
19    return x

```

Листинг 2: Полиномиальная аппроксимация МНК (общий случай)

```

1 def lsq_polynomial(xs, ys, degree):
2     n = len(xs)
3     m = degree
4     A = [[0.0] * (m + 1) for _ in range(m + 1)]
5     b = [0.0] * (m + 1)
6     for i in range(m + 1):
7         for j in range(m + 1):
8             A[i][j] = sum(xs[k] ** (i + j) for k in range(n))
9             b[i] = sum(ys[k] * xs[k] ** i for k in range(n))
10    return gauss_solve(A, b)
11
12 def poly_eval(coeffs, x):
13    return sum(coeffs[i] * x ** i for i in range(len(coeffs)))

```

Листинг 3: Линеаризация нелинейных функций

```

1 def lsq_exponential(xs, ys):
2     # phi = a * e^(b*x) => ln(y) = ln(a) + b*x
3     filtered = [(x, y) for x, y in zip(xs, ys) if y > 0]
4     xs_f = [p[0] for p in filtered]
5     ys_f = [math.log(p[1]) for p in filtered]
6     coeffs = lsq_polynomial(xs_f, ys_f, 1)
7     return math.exp(coeffs[0]), coeffs[1]
8
9 def lsq_power(xs, ys):
10    # phi = a * x^b => ln(y) = ln(a) + b*ln(x)
11    filtered = [(x, y) for x, y in zip(xs, ys) if x > 0 and y > 0]
12    xs_f = [math.log(p[0]) for p in filtered]
13    ys_f = [math.log(p[1]) for p in filtered]
14    coeffs = lsq_polynomial(xs_f, ys_f, 1)
15    return math.exp(coeffs[0]), coeffs[1]
16
17 def lsq_logarithmic(xs, ys):
18    # phi = a*ln(x) + b => X = ln(x), Y = y
19    filtered = [(x, y) for x, y in zip(xs, ys) if x > 0]
20    xs_f = [math.log(p[0]) for p in filtered]
21    ys_f = [p[1] for p in filtered]
22    coeffs = lsq_polynomial(xs_f, ys_f, 1)
23    return coeffs[1], coeffs[0] # a, b

```

Листинг 4: Метрики качества аппроксимации

```

1 def compute_metrics(xs, ys, phi_vals):
2     n = len(xs)
3     eps = [phi_vals[i] - ys[i] for i in range(n)]

```



```

4     S = sum(e ** 2 for e in eps)
5     delta = math.sqrt(S / n)
6     phi_mean = sum(phi_vals) / n
7     ss_res = sum((ys[i] - phi_vals[i]) ** 2 for i in range(n))
8     ss_tot = sum((ys[i] - phi_mean) ** 2 for i in range(n))
9     R2 = 1 - ss_res / ss_tot if ss_tot > 1e-15 else 0.0
10    return S, delta, R2, eps
11
12    def pearson_r(xs, ys):
13        n = len(xs)
14        x_mean = sum(xs) / n
15        y_mean = sum(ys) / n
16        num = sum((xs[i] - x_mean) * (ys[i] - y_mean) for i in range(n))
17        den = math.sqrt(sum((xs[i] - x_mean) ** 2 for i in range(n)) *
18                        sum((ys[i] - y_mean) ** 2 for i in range(n)))
19        return num / den if den > 1e-15 else 0.0

```

4.3 Результаты выполнения программы

4.3.1 Тест 1: Данные варианта 2

Запуск с данными варианта 2 ($n = 11$ точек, $x \in [0,4]$, $h = 0,4$). Результаты сравнения всех функций:

Функция	S	δ	R^2	Качество
Линейная	21.473987	1.397205	0.306409	недостаточно
Квадратичная	16.718797	1.232838	0.459997	недостаточно
Кубическая	3.967599	0.600575	0.871850	удовлетворительно
Экспоненциальная	84.635669	2.773833	-1.26	недостаточно
Логарифмическая	9.561484	0.977828	0.652276	слабая
Степенная	40.290762	2.007256	-0.47	недостаточно
Наилучшая аппроксимация: Кубическая ($\delta = 0,601$)				

Таблица 5: Сравнение аппроксимаций, вариант 2

Кубическая аппроксимация: $\varphi(x) = 0,4905 + 7,6238x - 4,7246x^2 + 0,7099x^3$.

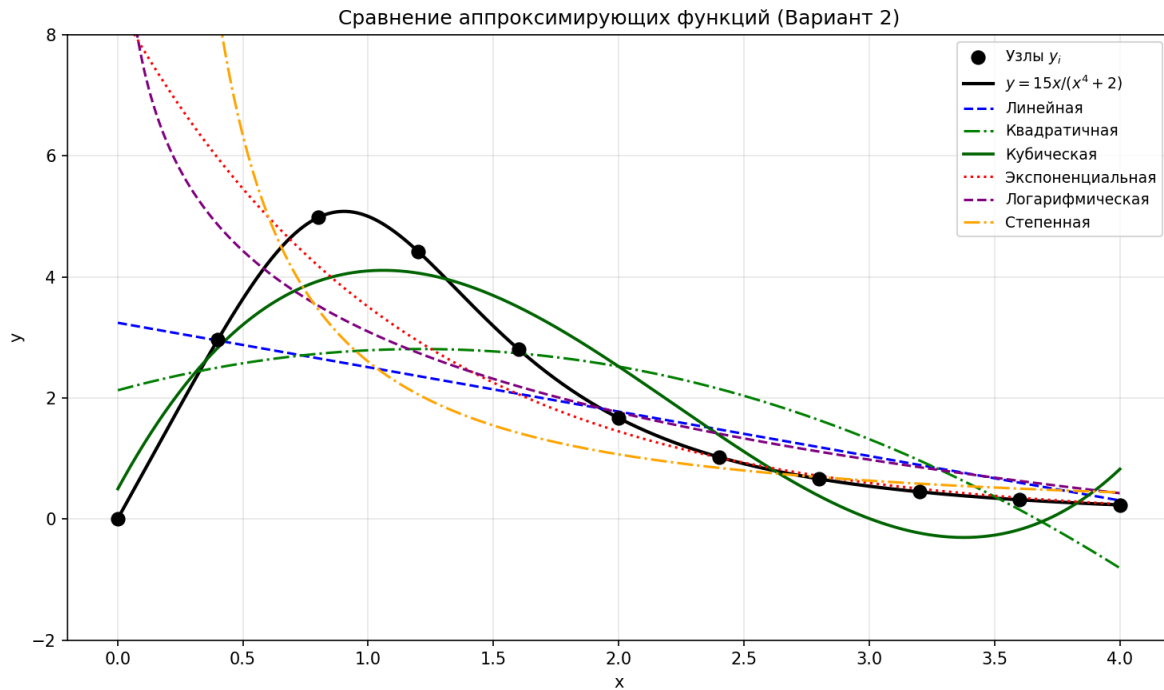


Рис. 2: Все аппроксимирующие функции для варианта 2

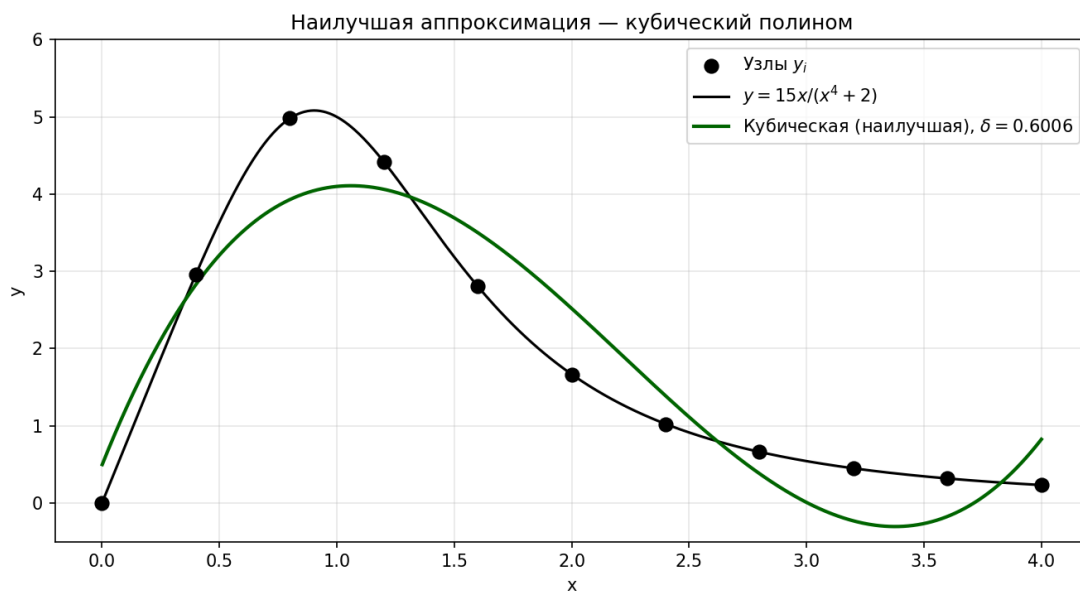


Рис. 3: Наилучшая аппроксимация — кубический полином

4.3.2 Тест 2: Линейные данные

Данные: $\{(1, 2), (2, 4), (3, 6), (4, 8), (5, 10), (6, 12), (7, 14), (8, 16)\}$ — точная линейная зависимость.

Ожидаемый результат: линейная функция $\varphi(x) = 2x + 0$ даёт $\delta = 0$, $R^2 = 1$. Программа выводит: $a = 2,0000$, $b = 0,0000$, $\delta \approx 0,000000$, $R^2 \approx 1,000000$. Коэффициент корреляции Пирсона: $r = 1,0000$ (строгая линейная связь).

4.3.3 Тест 3: Некорректные данные

При попытке аппроксимировать экспоненциальной функцией данные с отрицательными значениями y , программа выводит сообщение:

Ошибка: Недостаточно точек с $y > 0$ для экспоненциальной аппроксимации и продолжает работу.

5 Выводы

1. Для функции $y = 15x/(x^4 + 2)$ на отрезке $[0, 4]$ был построен ряд аппроксимирующих функций методом наименьших квадратов.
2. Линейная и квадратичная аппроксимации показали неудовлетворительные результаты ($R^2 < 0,5$), так как исходная функция имеет выраженный максимум в районе $x \approx 0,8$ и монотонно убывает к нулю — такое поведение невозможно описать прямой или обычной параболой.
3. Наилучшим приближением оказался **кубический полином** с $\delta = 0,601$ и $R^2 = 0,872$ (удовлетворительная аппроксимация).
4. Экспоненциальная и степенная функции показали худшие результаты ($R^2 < 0$), что объясняется монотонным характером этих функций, не способных воспроизвести «горб» исходных данных.
5. Разработанное консольное приложение на Python реализует метод наименьших квадратов для 6 классов функций с самостоятельно написанными алгоритмами (без `numpy.polyfit`), поддерживает ввод данных из консоли и файла, вычисляет все требуемые метрики и строит графики.